



Acceleration of Pattern Matching Algorithms Using Parallel Programming

Matheus Antônio de Castro de Barros

Orientador: Prof. Jeremias Moreira Gomes

Engenharia de Software

Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa

Roteiro

1

Introdução

2

**Revisão da
literatura**

3

Metodologia

4

Resultados

5

Conclusão

1

Introdução

1 · Introdução

2 · Revisão

3 · Metodologia

4 · Resultados

5 · Conclusão

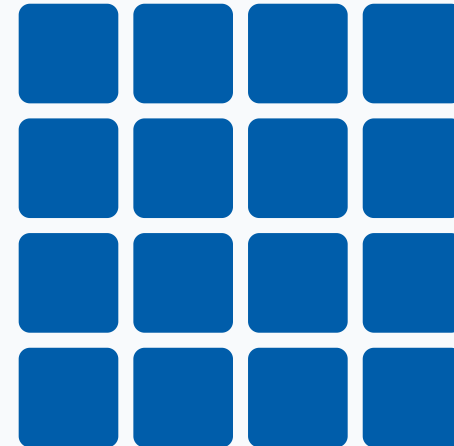
CPUs multinúcleo

Mudanças de arquitetura — o desempenho passou a vir de mais núcleos no mesmo chip.

Até anos ~2000: um núcleo cada vez mais rápido



Hoje: muitos núcleos — ex.: 16 núcleos / 32 threads



Um algoritmo sequencial usa **um núcleo de cada vez** — os demais ficam ociosos.

O casamento de padrões

Procurar **um ou múltiplos padrões** em conjuntos de dados que podem ter gigabytes

Antivírus

Assinaturas de malware procuradas em arquivos e na memória.

Detecção de intrusão

Regras conhecidas aplicadas ao tráfego de rede.

Bioinformática

Sequências de interesse localizadas em genomas inteiros.

Bancos de dados

Filtros textuais como LIKE '%...%' avaliados linha a linha, varrendo tabelas inteiras.

O casamento de padrões

Dado um texto **T** e um padrão **P**, encontrar todas as posições em que **P** ocorre dentro de **T**.

T = "ushers"

u	s	h	e	r	s
---	---	---	---	---	---

P = "her" — ocorrência na posição 3

O casamento de padrões

Algoritmo ingênuo

Tentar o padrão em cada posição do texto, comparando caractere a caractere.

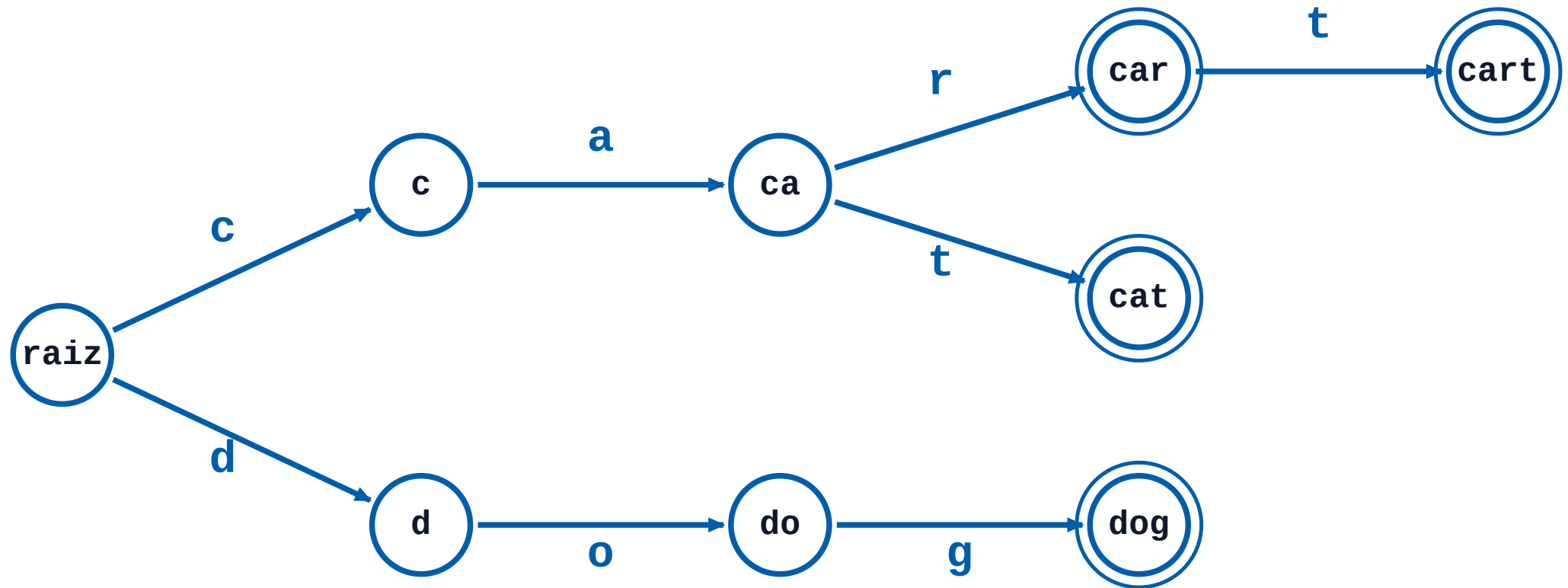
T = "ushers"

u	s	h	e	r	s
---	---	---	---	---	---

Tries

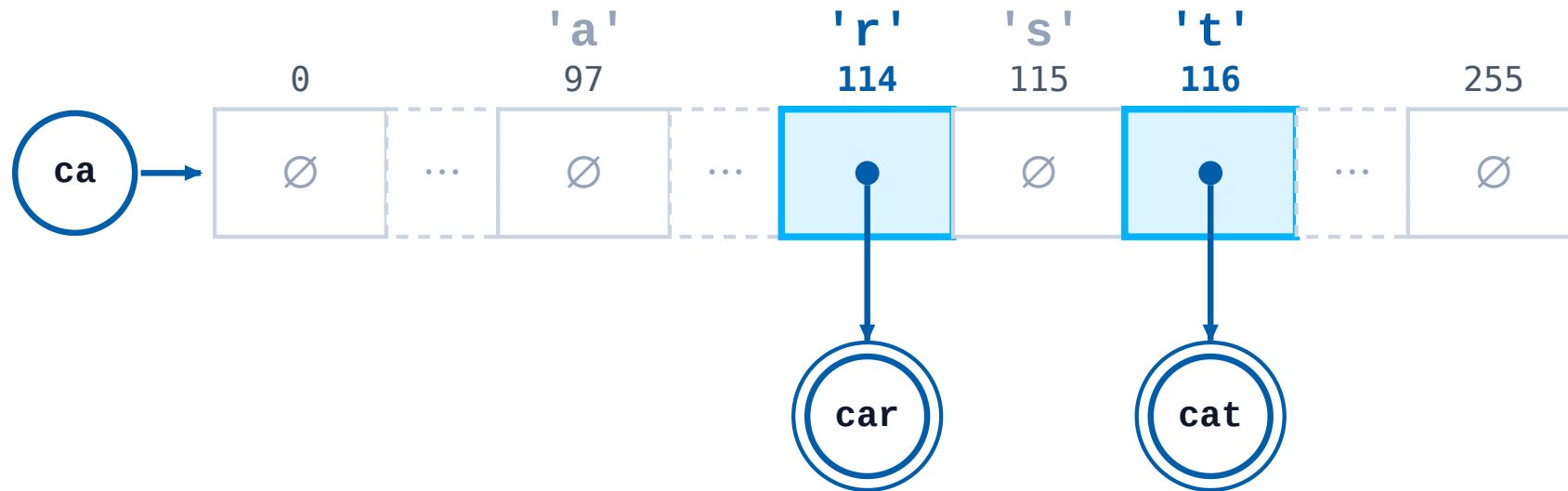
Trie (árvore de prefixos) é uma estrutura que armazena strings compartilhando os prefixos em comum.

Palavras do exemplo: `car` `cat` `cart` `dog`



Tries

Ponteiros representam caracteres.

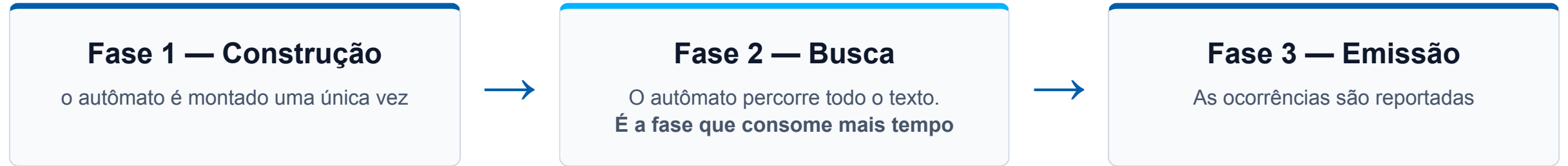


posições 'r' e 't' guardam
ponteiros para os próximos
estados; as 254 restantes
ficam vazias (∅)

O caractere não é armazenado: ele é a **própria posição** no arranjo.

Aho-Corasick

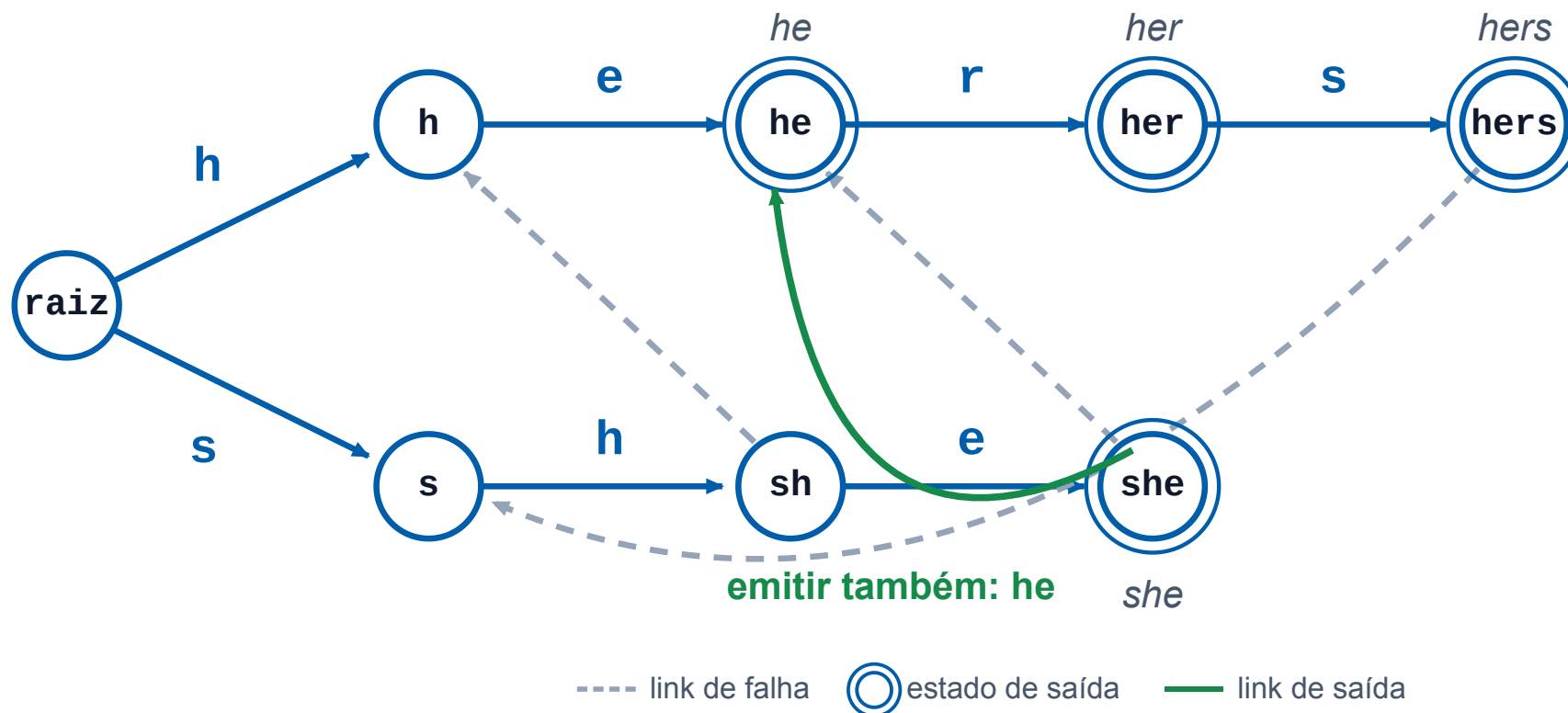
A execução é organizada em três fases



Aho-Corasick

Fase 1 — a trie ganha três acréscimos: **links de falha**, **estados de saída** e **links de saída**.

Padrões: he her hers she

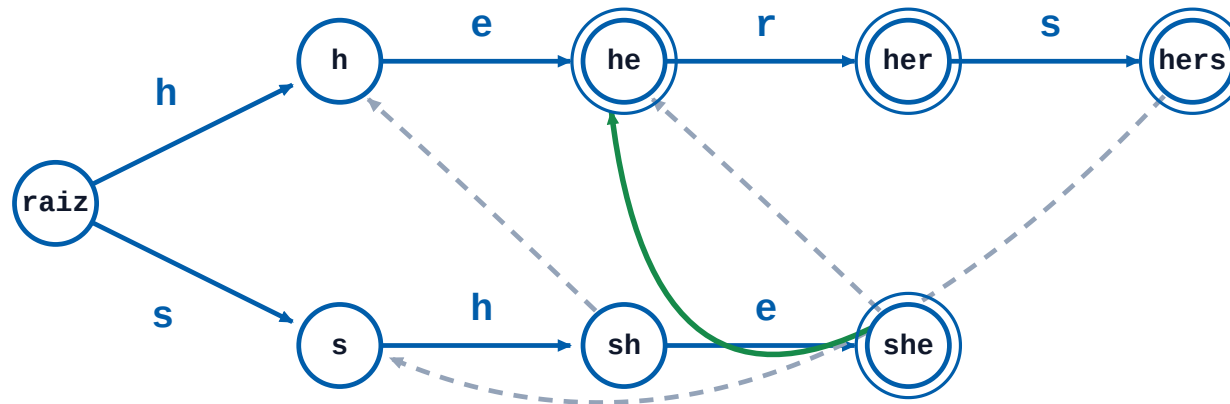


Aho-Corasick

Fases 2 e 3 — Varredura do texto

ushers

u s h e r s



Matches:

she

he

her

hers

2

Revisão da literatura

1 · Introdução

2 · Revisão

3 · Metodologia

4 · Resultados

5 · Conclusão

Revisão sistemática da literatura

Entender como o Aho-Corasick vem sendo paralelizado e otimizado.

196 registros recuperados
nas quatro bases, com as duas frentes de busca

24 estudos únicos pré-selecionados
após remoção de duplicatas e triagem de título e resumo

10 estudos primários
revisados em detalhe

Web of Science

Scopus

ACM Digital Library

IEEE Xplore

Lacunas identificadas

Trabalhos correlatos estudam:

Abordagem em CPUs

Em CPU, há precedentes de paralelismo por dados: dividir a **entrada** entre workers e tratar fronteiras.

Qu et al. (MPI); Tovarňák (REtrie)

Só a fase de busca

O paralelismo recai apenas sobre a **busca no texto**: construir o autômato e reportar as ocorrências seguem **sequenciais**.

Consenso nos 24 estudos revisados

Abordagens diferentes

GPU, FPGA, SIMD e autômatos modificados atacam **outros eixos**.

Thambawita et al. (GPU); Deyannis et al. (FPGA)

Lacuna e proposta

- ▶ Utilizar POSIX Threads em memória compartilhada, sem modificar o autômato.
- ▶ Otimizar as três fases do algoritmo.

3

Metodologia

1 · Introdução

2 · Revisão

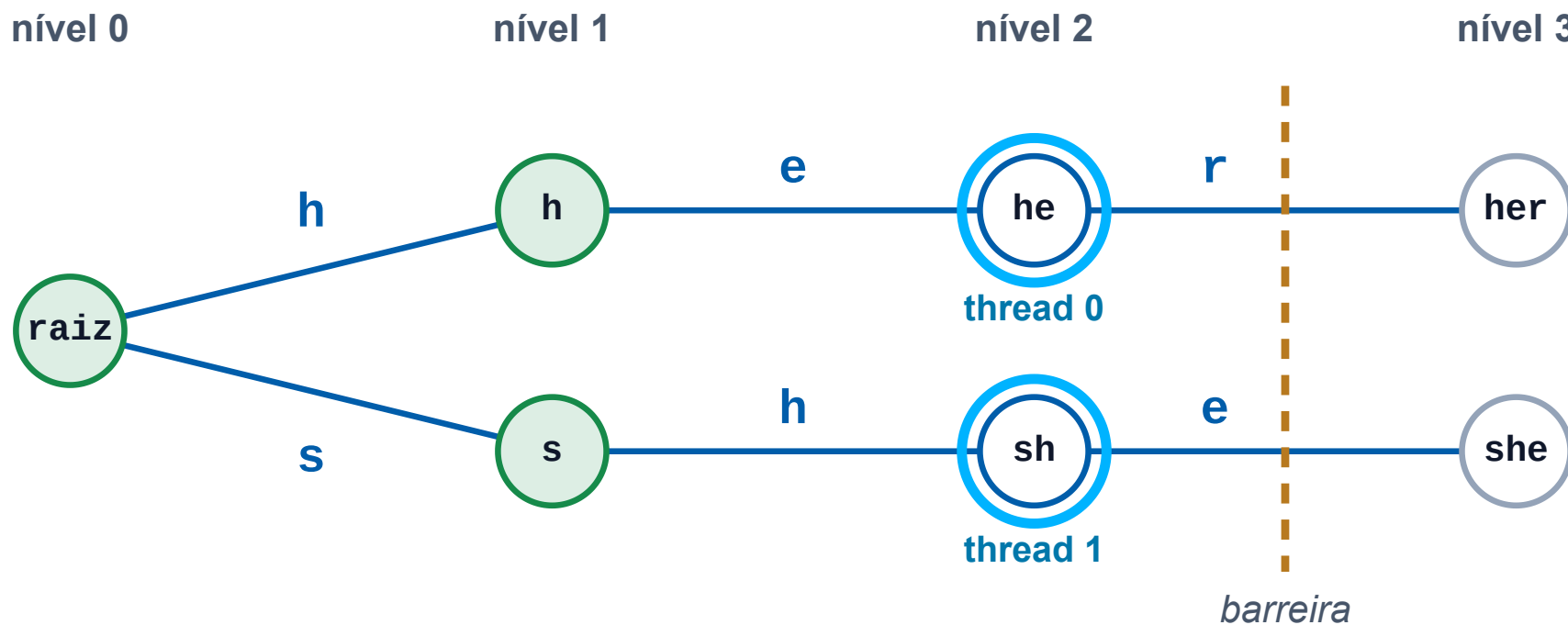
3 · Metodologia

4 · Resultados

5 · Conclusão

Fase 1: construção paralela

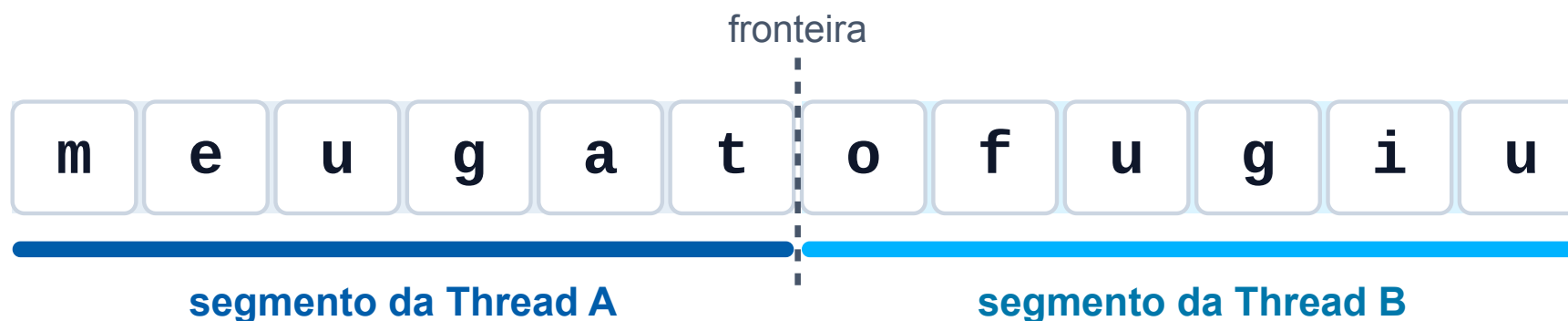
A inserção dos padrões na trie é sequencial; o cálculo dos **links de falha** é paralelizável, nível a nível.



O link de falha de um estado depende só de níveis **mais rasos** — estados do mesmo nível são independentes.

Fase 2: busca — Estratégia 1

Partição estática do texto: segmentos fixos, um por thread
— sobreposição na fronteira para não perder matches.



Exemplo: o padrão mais longo do dicionário é **gato** · texto dividido entre duas threads

Cada match pertence ao segmento onde **termina**.

Fase 2: busca — Estratégia 2

Despacho dinâmico: o texto é dividido em muitas tarefas pequenas — quem termina puxa a próxima tarefa livre.

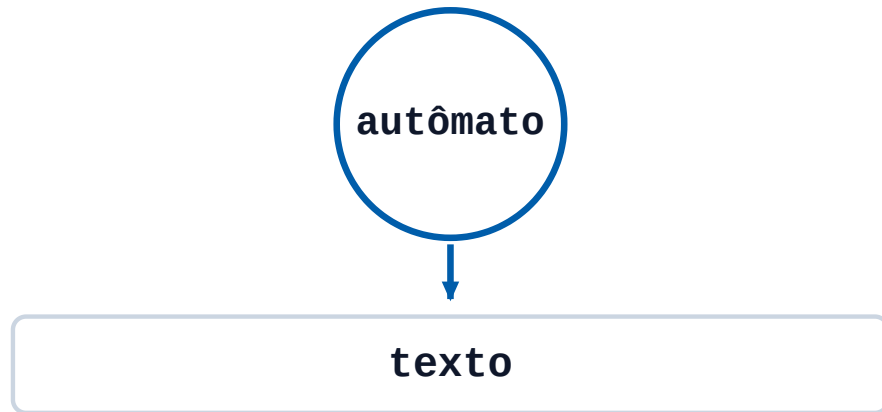


Na partição estática, quem recebe o trecho lento termina por último — as demais threads **esperam**.

Fase 2: busca — Estratégia 3

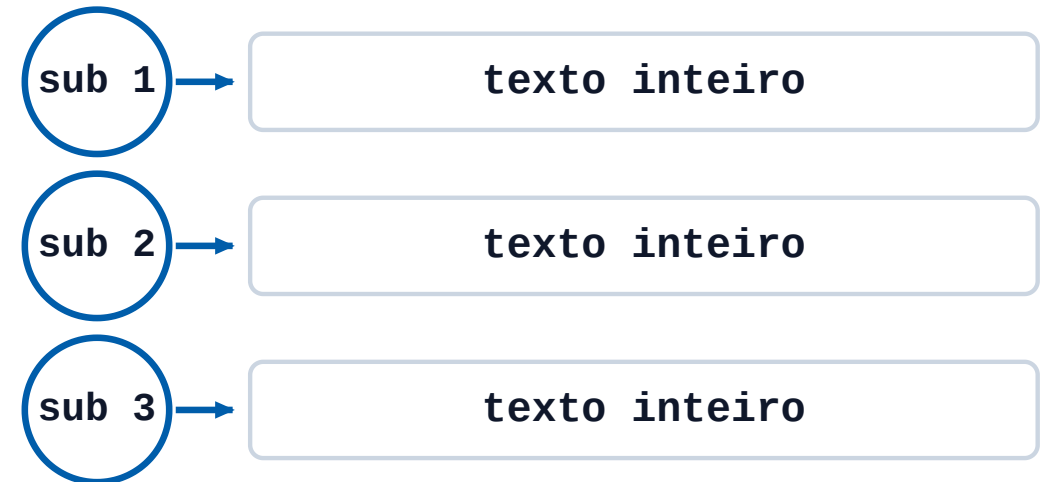
Partição do dicionário: os padrões são repartidos em K grupos, gerando K sub-autômatos menores.

Um autômato completo



lê o texto 1 vez

K = 3 sub-autômatos

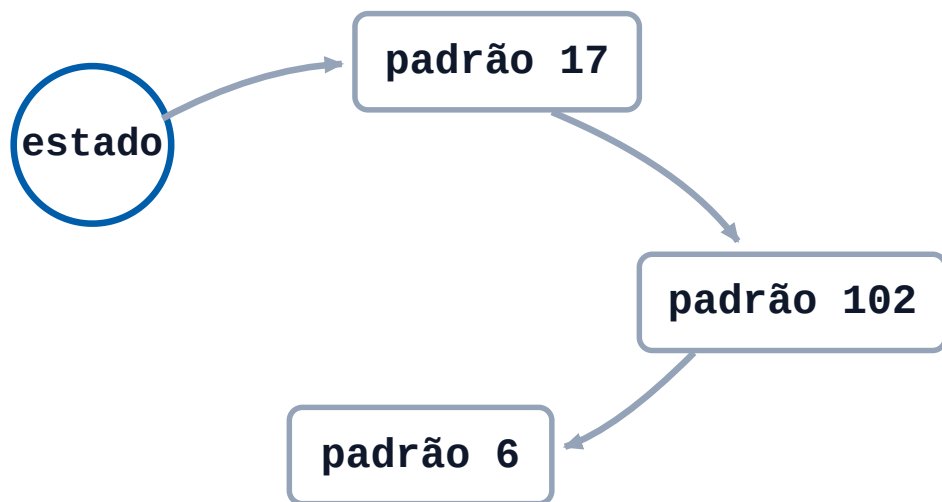


cada um relê o texto inteiro — leituras $\times K$

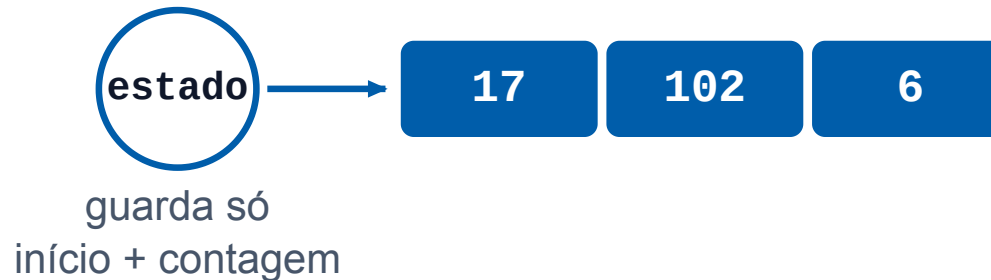
Fase 3: reportar ocorrências

Saída plana: ao alcançar um estado final, o autômato canônico percorre uma cadeia de ponteiros — a saída plana transforma em um bloco contíguo.

Layout canônico — ponteiros



Saída plana — bloco contíguo



4

Resultados

1 · Introdução

2 · Revisão

3 · Metodologia

4 · Resultados

5 · Conclusão

Ambiente de medição

Processador — AMD Ryzen 9 9950X



16 núcleos · **32 threads**

Hierarquia de memória

CCD 0 — 8 núcleos

CCD 1 — 8 núcleos

Memória principal DDR5 123 GiB

Código em C · POSIX Threads · Linux

Núcleos híbridos aparecem **em testes complementares**

Conjuntos de dados

Dicionários derivados de regras reais de detecção de intrusão.

Snort-100 · 100 padrões
1.939 estados | **1,93 MiB**

Snort-1k · ~1.000 padrões
11.974 estados | **11,92 MiB**

Snort completo · 4.188 padrões
55.479 estados | **55,23 MiB**

Emerging Threats · 44.678 padrões
508.896 estados | **506,78 MiB**

Corpora de entrada — gigabytes de texto real:

Enron
1,32 GiB

corpus principal para os resultados

Enron 8×
10,59 GiB

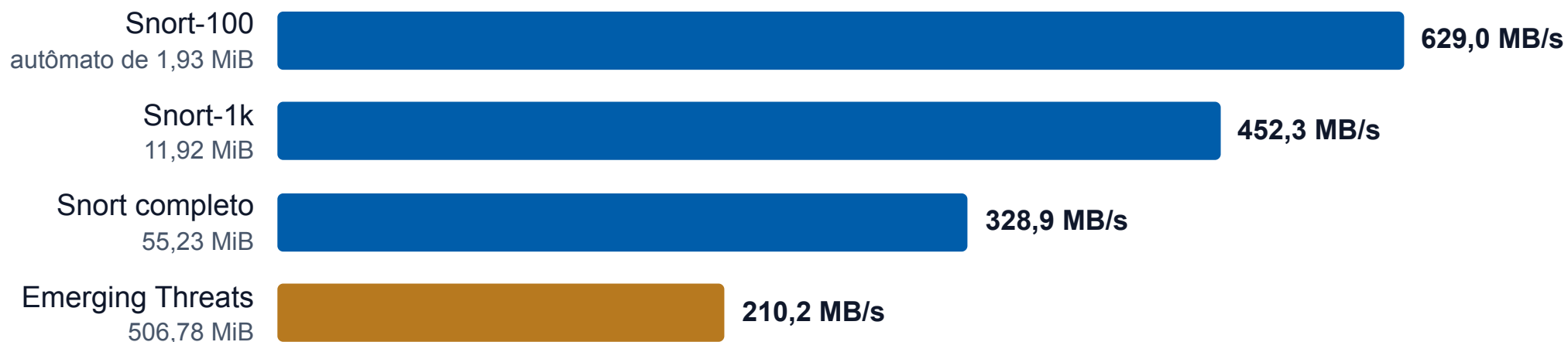
réplica óctupla.

SimpleWiki
1,19 GiB

testar o efeito do conteúdo

Resultados

Tamanho do autômato — execução **sequencial**: só o dicionário varia e o throughput cai **-66,6%**.



Trocar só o conteúdo do corpus (SimpleWiki → Enron) muda o throughput em ~5%

Resultados

Fase 1 — Construção paralela: compensa quando o autômato é **grande**.

1,59×

Emerging Threats · 508.896 estados
324 → 203 ms em 16 threads

1,17×

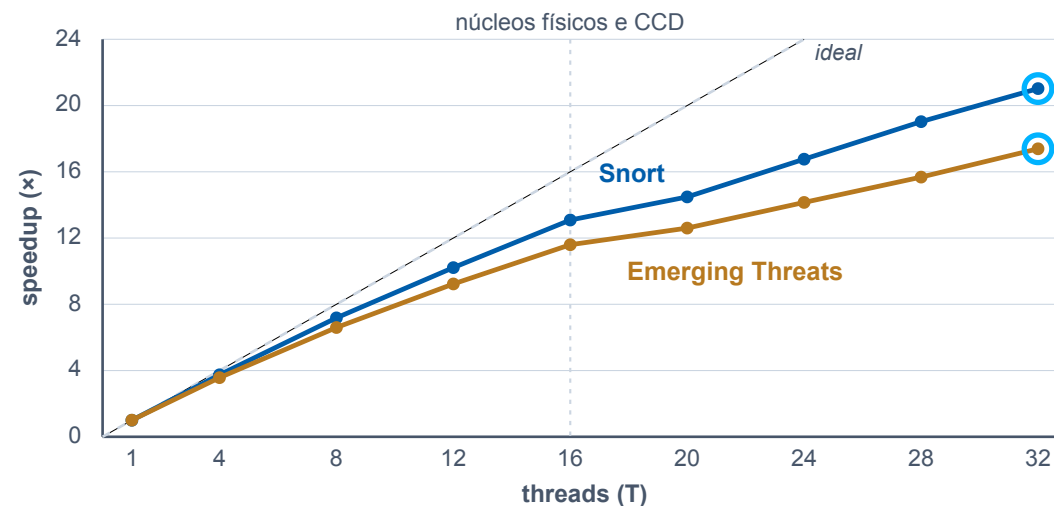
Snort completo · 55.479 estados
28 → 24 ms em 4 threads

0,70×

Snort-100 · 1.939 estados
0,8 → 1,1 ms em 2 threads

Resultados

Fase 2 — Estratégia 1 (partição estática), corpus Enron: escalabilidade até 32 threads



21,02×

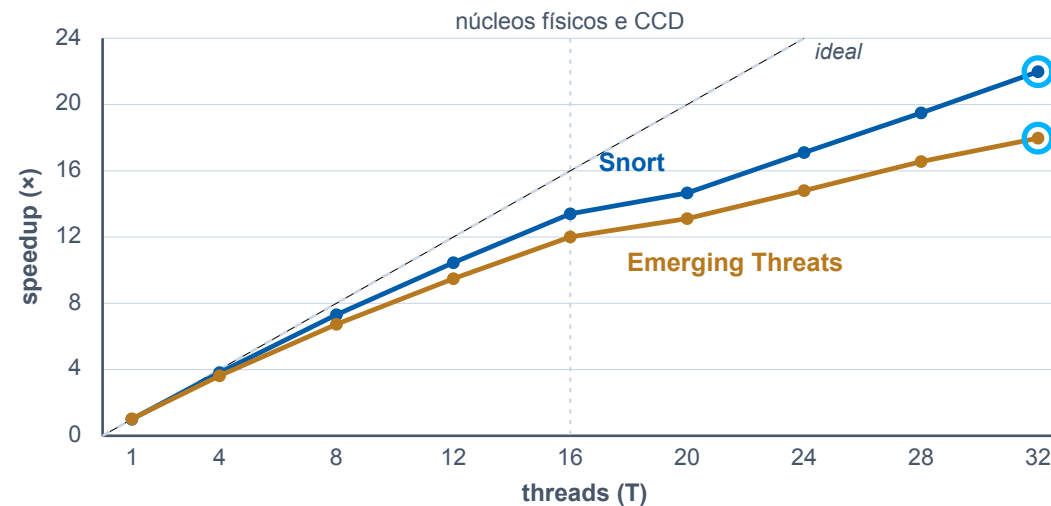
Snort · **6.920 MB/s** · eficiência de 65,7% em 32 threads

17,39×

Emerging Threats · **3.649 MB/s** · eficiência de 54,3% em 32 threads

Resultados

Fase 2 — Estratégia 2 (despacho dinâmico), corpus Enron: escalabilidade até 32 threads



21,98×

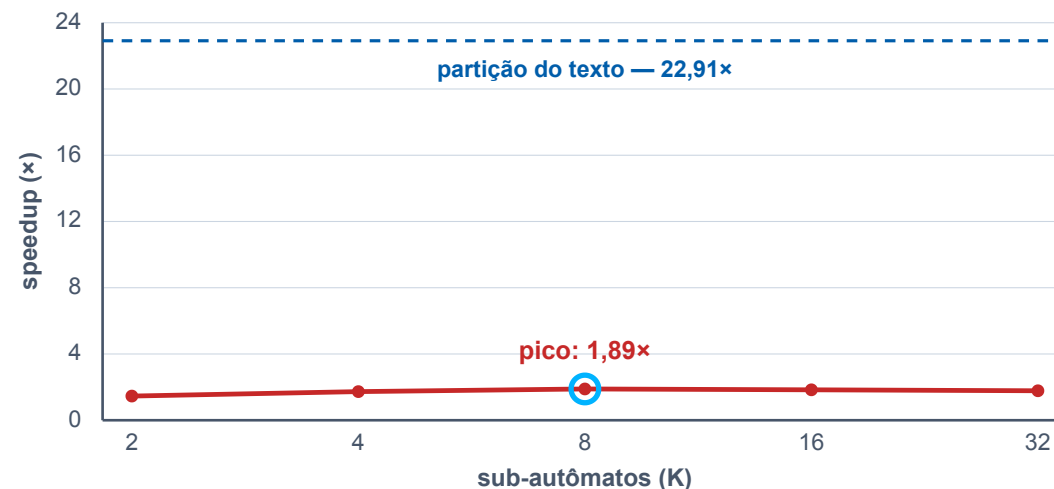
Snort · **7.236 MB/s** · eficiência de 68,7% em 32 threads

17,97×

Emerging Threats · **3.771 MB/s** · eficiência de 56,2% em 32 threads

Resultados

Fase 2 — Estratégia 3 (partição do dicionário), dicionário Snort, corpus Enron.



1,89x

pico com **8 sub-autômatos** (620,9 MB/s); adicionar mais grupos **piora**:
1,78x com 32

22,91x

partição do texto no mesmo cenário — cada sub-autômato relê o texto
inteiro, e o tráfego de memória domina

Resultados

Fase 3 — Saída plana: ganho medido em execução sequencial, corpus Enron.

+3,0%

Snort-100
629,0 → 647,8 MB/s

+4,2%

Snort completo
328,9 → 342,6 MB/s

+6,9%

Emerging Threats
210,2 → 224,6 MB/s

Os melhores resultados do trabalho combinam despacho dinâmico com saída plana, **22,91×** e **18,96×**.

5

Conclusão

1 · Introdução

2 · Revisão

3 · Metodologia

4 · Resultados

5 · Conclusão

Conclusão

Quatro achados relevantes

O tamanho do autômato tem grande impacto

Condicionado pela memória, mesmo com 32 threads.

Construção impacta autômatos grandes

Paralelizar a fase 1 compensa apenas para autômatos grandes.

Particionar texto é mais eficiente

22,91× (Snort) e **18,96×** (Emerging Threats) em 32 threads.
Particionar o dicionário gera ganho pequeno.

O layout de emissão importa

A emissão plana corta a perseguição de ponteiros e traz ganhos a custo mínimo.

Trabalhos futuros

Limitação reconhecida

Causas de cache e DRAM **inferidas** — sem contadores de hardware

Rodada de testes única

Impacto do conteúdo concentrado aferido apenas em plataforma híbrida



Trabalho futuro

Medir direto: contadores de desempenho (misses, banda, stalls)



Réplicas independentes, com mediana e dispersão



Repetir o diagnóstico na plataforma homogênea

Obrigado

Matheus Antônio de Castro de Barros

Orientador: Prof. Jeremias Moreira Gomes

Engenharia de Software · IDP — Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa

Material extra

Fase 2 — núcleos desiguais

Variação de tempo entre iterações.

i5 híbrido · 12 threads

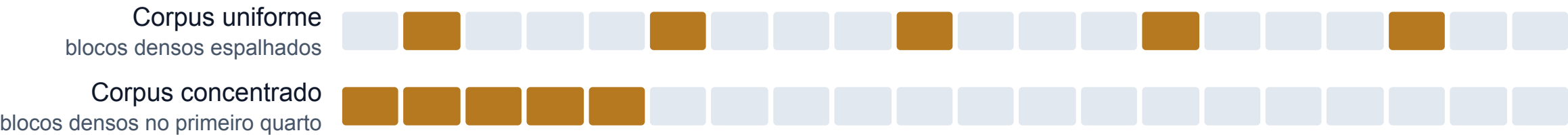
Estática simples	39,9%
Estática por topologia	1,7%
Despacho dinâmico	3,3%

Ryzen homogêneo · 32 threads

Estática simples	4,1%
Estática por topologia	0,6%
Despacho dinâmico	0,7%

Fase 2 — conteúdo concentrado

Mesmos bytes e ocorrências; muda onde o trabalho fica.



Estratégia	Thread mais lenta	Thread mais rápida
Estática simples	11,76 s	3,71 s
Estática por topologia	11,44 s	3,07 s
Despacho dinâmico	10,52 s	10,16 s